CODE EXPLANATION AND 5-MINUTE VIDEO CRAM GUIDE

VoiceToNote

What the code does, why it is written that way, and how to explain it clearly

This document is written for one purpose: to make you ready to open the code and explain it with confidence. It covers the architecture, every important file, the security decisions, the AWS terms you are likely to be asked about, and a timed script for a five-minute walkthrough.

Project	VoiceToNote - voice recording to structured notes
Stack	React and Vite, Cognito, API Gateway HTTP API, Lambda, S3, Transcribe, EventBridge, Bedrock, DynamoDB
Region	af-south-1 (Cape Town), with Bedrock in us-east-1
Design goal	Zero idle cost, event-driven processing, user-scoped data, least privilege
Video length	5 minutes - script starts in Section 9

How to use this document

- Read Sections 1 to 3 first. They give you the story and the end-to-end flow.
- Read Sections 4 to 6 with the code open. Each file has a what, a why, and a security note.
- Use Section 7 as flash cards. If you can answer those questions, you understand the project.
- Use Section 9 on the day. It is the exact order and the exact points to say in the video.

The one-sentence version: *VoiceToNote is a serverless voice-note app where the browser records audio, uploads it directly to S3 using a short-lived signed URL, Transcribe converts speech to text, Bedrock cleans and structures the text, and DynamoDB stores the result per user behind a Cognito-protected API.*

1. The project in one page

VoiceToNote solves a simple problem: people speak faster than they type. The app lets a signed-in user record a voice note, then turns that recording into a clean note with a title, a one-sentence summary, a cleaned transcript, a status, and a list of tasks.

The user story

- The user creates an account or signs in. Cognito returns tokens.
- The user presses Rec. The browser records audio with the MediaRecorder API.
- The browser asks the API for an upload URL, then uploads the audio directly to S3.
- AWS Transcribe converts the audio to text in the background.
- Bedrock cleans the transcript and returns structured JSON.
- The note is saved in DynamoDB and appears in the user's list.
- The user can edit the transcript, tick tasks, change status, or delete the note.

The engineering story

- **No server to manage.** Every compute unit is a Lambda function. There is no EC2 instance and no container running all day.
- **No idle cost.** DynamoDB is on-demand, Lambda charges per request, and Transcribe and Bedrock charge per use. With no traffic, the bill is storage only.
- **Direct upload.** Audio goes from the browser to S3, not through Lambda. This avoids API Gateway and Lambda payload limits and keeps the upload fast.
- **Event-driven.** S3 triggers Transcribe; EventBridge triggers processing. The browser does not sit in a long request waiting for AI.
- **User isolation.** The user ID comes from the validated JWT and is the DynamoDB partition key, so a user cannot request another user's notes.
- **Graceful failure.** If Bedrock fails, the raw transcript is still saved. The note is less polished, but the user's data is not lost.

The five design decisions to remember

Decision	Why it matters
Upload directly to S3 with a presigned URL	Keeps AWS credentials out of the browser and avoids sending binary data through Lambda.
Use Cognito and a JWT authorizer	Authentication is delegated to AWS. API Gateway rejects unauthenticated requests before Lambda runs.
Use userId as the DynamoDB partition key	Every read and write is naturally scoped to one user. Tenant isolation is built into the data model.
Use asynchronous Transcribe plus EventBridge	Speech-to-text and AI can take seconds. The request returns immediately and the UI polls.
Ask Bedrock for strict JSON	The frontend needs predictable fields, not free-form prose. Temperature 0 and a fixed schema reduce variation.

In one breath: record, upload to S3, transcribe, structure with Bedrock, save per user behind Cognito and API Gateway.

2. Architecture at a glance

The system has one frontend, one API, four application Lambda functions, and seven managed AWS services. The important part is not the number of services. The important part is the direction of the arrows: the browser only talks to the API and to one short-lived S3 URL. Everything else happens inside AWS.

Component	Responsibility	Why it is used
React + Vite	Recording UI, notes list, detail view, auth screens	Fast static frontend that can be hosted on S3 or any static host.
Cognito User Pool	Sign-up, sign-in, email verification, JWT issuance	Managed identity. No custom password database or crypto.
API Gateway HTTP API	Routes, CORS, JWT validation	Cheaper and simpler than REST API for this use case. Native JWT authorizer.
Lambda: get_upload_url	Creates a presigned S3 PUT URL	Only this function needs S3 write permission.
S3 audio bucket	Stores audio files privately	Durable object storage with lifecycle rules and event notifications.
Lambda: start_transcription	Starts an AWS Transcribe job	Keeps transcription asynchronous and outside the request path.
AWS Transcribe	Speech-to-text	Managed ASR. No model hosting or audio processing code.
EventBridge	Delivers the Transcribe completion event	Decouples completion from the function that started the job.
Lambda: process_transcript	Downloads transcript, calls Bedrock, writes DynamoDB	Orchestrates the post-processing pipeline.
Amazon Bedrock (Nova Micro)	Cleans text and extracts title, summary, status, todos	Managed LLM. No GPU infrastructure.
DynamoDB	Stores notes per user	On-demand, single-digit millisecond reads, zero idle capacity.
Lambda: notes_api	List, get, update, delete notes	One small CRUD API with the same auth model.

The data model

The DynamoDB table is deliberately simple. It has a composite primary key: `userId` as the partition key and `noteId` as the sort key. That means all notes for one user live together, and a query for one user never touches another user's data.

```
DynamoDB item
  userId      string   partition key   (Cognito sub claim)
  noteId      string   sort key        (server-generated UUID)
  title       string
  summary     string
  text        string                  cleaned transcript
  rawText     string                  original transcript
  status      string                  "Planned" or "Done"
  todos       list of { text, done }
  createdAt   string                  ISO-8601 UTC
  updatedAt   string                  ISO-8601 UTC
  audioKey    string                  audio/<userId>/<noteId>.<ext>
```

The S3 key is also a data structure

The key `audio/<userId>/<noteId>.webm` is not random decoration. It lets the S3 event handler recover the owner and the note ID without a database lookup. It also makes the bucket easy to reason about: everything for one user sits under one prefix.

The high-level flow

- **Identity:** the browser authenticates with Cognito and stores the returned tokens.
- **Authorisation:** every API request carries the ID token as a Bearer token. API Gateway validates signature, issuer, audience, and expiry.
- **Ingestion:** the API returns a presigned S3 URL. The browser PUTs the audio blob to that URL.
- **Transcription:** S3 emits an ObjectCreated event. A Lambda starts a Transcribe job.
- **Structuring:** EventBridge emits a completion event. A Lambda downloads the transcript and calls Bedrock.
- **Persistence:** the structured note is written to DynamoDB under the user's partition.
- **Presentation:** the React app polls the notes API while processing, then renders the finished note.

Key line for the video: the browser never holds AWS credentials, and it never talks to DynamoDB, Transcribe, or Bedrock directly. It only holds a Cognito token and a five-minute S3 URL.

3. End-to-end flow, step by step

Being able to narrate this sequence is more important than memorising every line of code. If you can explain the flow and the reason for each hop, you can answer most questions.

A. Sign-in and token issue

- The frontend sends the email and password to the Cognito user pool using the InitiateAuth API.
- Cognito returns an ID token, an access token, and a refresh token.
- The app stores them in localStorage. The ID token is a signed JWT that contains claims such as sub, email, iss, aud, and exp.
- The ID token is sent as `Authorization: Bearer <idToken>` on API requests.

B. Recording and upload

- The browser calls `navigator.mediaDevices.getUserMedia({ audio: true })` and receives a media stream.
- A MediaRecorder records the stream as `audio/webm` and collects data chunks.
- When the user stops, the chunks are combined into a Blob.
- The app calls `POST /upload-url` with the file extension.
- The upload Lambda reads the user ID from the JWT, generates a UUID, and returns a presigned S3 URL, the note ID, and the key.
- The browser sends `PUT <presignedUrl>` with the audio Blob and `Content-Type: audio/webm`.
- S3 stores the object and emits an ObjectCreated event.

C. Transcription

- The S3 notification invokes `start_transcription` for keys under the `audio/` prefix.
- The function decodes the key, extracts the user and note IDs, and builds a Transcribe job name.
- It calls `start_transcription_job` with the S3 URI, media format, and language code.
- Transcribe runs asynchronously. The original request has already finished.
- When the job completes, EventBridge emits a Transcribe Job State Change event with status COMPLETED.

D. Structuring and saving

- The EventBridge rule invokes `process_transcript` with the job name.
- The function calls `get_transcription_job` to get the transcript URI.
- It downloads the transcript JSON and extracts the plain text.
- It calls Bedrock with a prompt that asks for cleaned text, a title, a summary, a status, and todos.
- It writes one DynamoDB item with the user ID and note ID as the key.
- If Bedrock fails, it still writes the raw transcript and a safe default status.

E. Reading, editing, and deleting

- The notes page calls `GET /notes`. The Lambda queries DynamoDB with the user ID from the token.

- The detail page reads from the local cache first, so it opens instantly, then updates after edits.
- Editing the transcript sends `PUT /notes/{id}` with the new text. The backend re-runs Bedrock so the title, summary, status, and todos stay consistent.
- Ticking a todo sends the updated todos and status. The UI updates immediately and rolls back if the request fails.
- Deleting sends `DELETE /notes/{id}` and also removes the item from the local cache.

Why the flow is split this way

Split	Reason
Browser to S3 directly	Binary uploads are faster and cheaper outside Lambda. The browser gets a narrow, expiring permission instead of AWS credentials.
Upload API separate from notes API	The upload function needs S3 PutObject. The notes function needs DynamoDB and Bedrock. Separate functions keep permissions small.
Transcribe separate from processing	Transcription is long-running and asynchronous. Processing only starts when the transcript exists.
EventBridge between Transcribe and Lambda	No polling loop, no open connection, and the completion event can be retried independently.
Polling in the UI	The user gets a processing card immediately. The frontend polls every five seconds for up to ninety seconds.

4. Infrastructure: template.yaml

The SAM template is the source of truth for the backend. It defines identity, storage, the API, the functions, the triggers, and the permissions. If you can explain this file, you can explain how the whole system is wired.

Global function settings

```
Globals:
  Function:
    Timeout: 30
    MemorySize: 256
    Architectures:
      - arm64
    Runtime: python3.12
```

- **arm64:** Graviton processors are cheaper per unit of compute for this workload. Python and boto3 run on ARM without changes.
- **256 MB:** enough for JSON, boto3, and a Bedrock response. More memory also gives proportionally more CPU, but this workload does not need it.
- **30 seconds:** a safe default. The two functions that call Transcribe or Bedrock override this to 60 seconds.
- **Python 3.12:** current supported runtime with good boto3 compatibility.

Cognito user pool and app client

```
UserPool:
  Type: AWS::Cognito::UserPool
  Properties:
    AutoVerifiedAttributes: [email]
    UsernameAttributes: [email]
    Policies:
      PasswordPolicy:
        MinimumLength: 8

UserPoolClient:
  Type: AWS::Cognito::UserPoolClient
  Properties:
    GenerateSecret: false
    ExplicitAuthFlows:
      - ALLOW_USER_PASSWORD_AUTH
      - ALLOW_REFRESH_TOKEN_AUTH
```

- **Why Cognito:** it gives us a managed user directory, email verification, password hashing, JWT signing, and token refresh without writing auth code.
- **Why email as the username:** it is the identifier users already know. Cognito still issues a stable `sub` claim, which is what the database uses.
- **Why no client secret:** a browser app cannot keep a secret. A public app client is the correct pattern for a SPA. Security comes from TLS, short-lived tokens, and the user's password.
- **Why USER_PASSWORD_AUTH:** it keeps the demo simple by sending the password directly to Cognito over TLS. In a production build I would use SRP or the hosted UI so the password is never handled by the app code.
- **Why REFRESH_TOKEN_AUTH:** it lets the frontend get a new ID token without asking the user to sign in again.

S3 bucket, CORS, lifecycle, and Transcribe access

```
AudioBucket:
  Type: AWS::S3::Bucket
```

```
  Properties:
    CorsConfiguration:
      CorsRules:
        - AllowedHeaders: ['*']
          AllowedMethods: [PUT, GET, HEAD]
          AllowedOrigins: ['*']
    LifecycleConfiguration:
      Rules:
        - Id: ExpireAudioAfter7Days
          Status: Enabled
          ExpirationInDays: 7
```

- **Why CORS:** the browser is on a different origin from S3. The PUT request needs CORS permission or the browser blocks it.
- **Why lifecycle 7 days:** audio is only needed long enough to transcribe and review. Expiring it limits storage cost and reduces the amount of personal data kept. The note text remains in DynamoDB.
- **Why the bucket is private:** there is no public read policy. Access is only through the signed upload URL and the services that need it.
- **Why a bucket policy for Transcribe:** the Transcribe service needs `s3:GetObject` on the audio object. The policy grants that to the Transcribe service principal, not to the public.
- **Why prefix-based notifications:** only `audio/` objects should start transcription. Other objects cannot accidentally trigger the pipeline.

DynamoDB table

```
NotesTable:
  Type: AWS::DynamoDB::Table
  Properties:
    BillingMode: PAY_PER_REQUEST
    AttributeDefinitions:
      - AttributeName: userId
        AttributeType: S
      - AttributeName: noteId
        AttributeType: S
    KeySchema:
      - AttributeName: userId
        KeyType: HASH
      - AttributeName: noteId
        KeyType: RANGE
```

- **Why on-demand:** there is no traffic forecast for a student project. PAY_PER_REQUEST scales automatically and charges only for reads and writes.
- **Why userId first:** the main access pattern is 'list my notes'. A query with the user ID reads one partition and cannot return another user's items.
- **Why noteId second:** it makes each note unique within a user and gives a direct key for get, update, and delete.
- **Why UUIDs:** they are generated on the server, so the client cannot choose or guess an ID, and collisions are practically impossible.

HTTP API and the JWT authorizer

```
HttpApi:
  Type: AWS::Serverless::HttpApi
  Properties:
    Auth:
      DefaultAuthorizer: CognitoAuthorizer
      Authorizers:
        CognitoAuthorizer:
          IdentitySource: "$request.header.Authorization"
          JwtConfiguration:
            audience: [!Ref UserPoolClient]
            issuer: !Sub https://cognito-idp.${AWS::Region}.amazonaws.com/${UserPool}
```

- **Why HTTP API:** it is cheaper and simpler than a REST API, and it has native JWT authorisation. We do not need REST API features such as request validators or usage plans.
- **Why DefaultAuthorizer:** every route is protected unless explicitly opted out. That is safer than remembering to add auth to each route.
- **Why audience and issuer:** the authorizer checks that the token came from this user pool and was issued for this app client. A token from another pool or another app is rejected.
- **What happens before Lambda:** API Gateway verifies the JWT signature, expiry, issuer, and audience. If any check fails, the request gets a 401 and no Lambda runs.
- **Where claims come from:** after validation, API Gateway places the claims in `requestContext.authorizer.jwt.claims`. The Lambda reads the user ID from there, never from the request body.

The S3 notification custom resource

This is the part most people cannot explain, so it is worth understanding. The bucket needs to notify a Lambda when an object is created. The Lambda needs permission for S3 to invoke it. The bucket notification references the Lambda ARN, and the permission references the Lambda. A normal SAM S3 event can create a circular dependency between the bucket, the permission, and the function. The template breaks that cycle with a small CloudFormation custom resource.

```
S3NotificationSetup:
  Type: AWS::CloudFormation::CustomResource
  DependsOn: StartTranscribeFunctionS3Permission
  Properties:
    ServiceToken: !GetAtt S3NotificationSetupFunction.Arn
    BucketName: !Ref AudioBucket
    LambdaArn: !GetAtt StartTranscribeFunction.Arn
```

- The custom resource waits for `StartTranscribeFunctionS3Permission` before it runs.
- Its inline Lambda calls `put_bucket_notification_configuration` and wires ObjectCreated events under `audio/` to the transcription function.
- On delete, it clears the notification so the bucket can be removed cleanly.
- The custom resource sends SUCCESS or FAILED back to CloudFormation using the presigned ResponseURL.
- **Why this is good to mention:** it shows you understand that infrastructure is code and that dependency order matters.

Least-privilege permissions

Function	Allowed actions	Why only those
get_upload_url	s3:PutObject on the audio bucket	It only signs uploads. It cannot read or delete audio.
start_transcription	s3:GetObject, transcribe:StartTranscriptionJob, transcribe:TagResource	It reads the object reference, starts one job, and tags it.
process_transcript	s3:GetObject, transcribe:GetTranscriptionJob, bedrock:InvokeModel, DynamoDB CRUD on NotesTable	It needs the transcript, the job status, the model, and the notes table.
notes_api	DynamoDB CRUD on NotesTable, bedrock:InvokeModel	It serves notes and re-runs AI when text is edited.

Interview-ready sentence: I did not give every function a wildcard policy. Each Lambda has only the actions it needs on the resources it needs. That limits the blast radius if one function is compromised.

5. Backend code, file by file

There are four handlers and one shared module. The handlers are intentionally small. Each one has a single job, and the shared AI module keeps the prompt and the Bedrock call in one place so the pipeline and the edit path behave the same.

5.1 get_upload_url.py - the signed upload

What it does

This function is the gatekeeper for uploads. It runs after API Gateway has already validated the JWT. It reads the authenticated user's ID from the token claims, creates a note ID, builds the S3 key, and returns a presigned URL.

```
authorizer = event.get("requestContext", {}).get("authorizer", {}).get("jwt", {})
claims = authorizer.get("claims", {})
user_id = claims.get("sub") or claims.get("username") or "anonymous"

body = json.loads(event.get("body", "{}") or "{}")
file_extension = body.get("extension", "webm")
note_id = str(uuid.uuid4())
s3_key = f"audio/{user_id}/{note_id}.{file_extension}"

presigned_url = s3_client.generate_presigned_url(
    "put_object",
    Params={"Bucket": AUDIO_BUCKET, "Key": s3_key, "ContentType": "audio/webm"},
    ExpiresIn=300,
)
```

Why it is written this way

- **The user ID is not taken from the request body.** If the browser could send a userId, an attacker could upload into another user's prefix. The ID comes from the signed token.
- **The note ID is generated on the server.** The client cannot choose a key or overwrite an existing note. `uuid.uuid4()` gives a random, collision-resistant ID.
- **The key encodes ownership.** `audio/<userId>/<noteId>.webm` lets the S3 event handler recover both IDs without a database read.
- **The URL expires in 300 seconds.** Five minutes is enough for a phone upload and short enough to limit replay if the URL leaks.
- **Only one operation is allowed.** The URL is for `put_object` on one key. It is not a general S3 credential.
- **ContentType is part of the signature.** The frontend must send exactly `audio/webm` or S3 returns SignatureDoesNotMatch.

SigV4 in plain English

SigV4 is AWS Signature Version 4. It is the protocol AWS uses to prove that a request was made by someone who holds valid AWS credentials, and that the request was not changed in transit. Instead of sending a password or secret key, the client uses the secret key to compute a signature over the request. AWS recomputes the same signature on its side and compares them.

What goes into the signature

- The HTTP method, for example PUT.
- The canonical URI, which is the bucket and object path.
- The canonical query string, including the signing parameters.
- The headers that are signed, including `host` and `content-type`.

- A hash of the request payload.
- The credential scope: access key ID, date, region, service, and `aws4_request`.

How a presigned URL works

- The Lambda has an IAM role. Boto3 gets temporary credentials for that role.
- Boto3 builds the request and computes the SigV4 signature using the secret access key.
- The signature and related values are placed in the query string: `X-Amz-Algorithm`, `X-Amz-Credential`, `X-Amz-Date`, `X-Amz-Expires`, `X-Amz-SignedHeaders`, and `X-Amz-Signature`.
- The browser sends that URL. S3 validates the signature, the expiry, the HTTP method, and the signed headers.
- If any signed value changes, the signature no longer matches and S3 rejects the request.

Why this design is better than uploading through Lambda

- **No AWS secrets in the browser.** The browser only receives a URL that expires. It never receives an access key or secret key.
- **No payload limit problem.** API Gateway and Lambda have request size limits. A direct S3 PUT can handle large audio files.
- **Lower cost and latency.** The audio does not pass through API Gateway or Lambda. S3 receives it directly.
- **Narrow permission.** The URL can only write one object, for five minutes. It cannot list, read, or delete anything else.

Why the explicit client configuration

```
s3_client = boto3.client(
    "s3",
    region_name=os.environ.get("AWS_REGION", "af-south-1"),
    config=Config(signature_version="s3v4", s3={"addressing_style": "virtual"}),
)
```

- `signature_version="s3v4"` forces Signature Version 4 instead of any legacy signing mode. S3 requires SigV4 in most regions, including newer ones.
- `addressing_style="virtual"` uses `bucket.s3.region.amazonaws.com/key` instead of putting the bucket in the path. Virtual-hosted style is the modern default and avoids path-style edge cases.
- The region must match the bucket region. A presigned URL signed for the wrong region will fail.

What SigV4 does not do

- It does not encrypt the audio. HTTPS encrypts the transport; S3 encryption at rest is separate.
- It does not make the bucket public. The URL is a temporary capability, not a public ACL.
- It does not replace authentication. The user still needs a valid Cognito token to get the URL.
- It does not last forever. After 300 seconds the URL is rejected.

5.2 start_transcription.py - S3 event to Transcribe

What it does

This function is triggered by S3 when a new audio object arrives. It extracts the user ID and note ID from the key, builds a unique job name, maps the file extension to a Transcribe media format, and starts an asynchronous transcription job.

```
for record in event.get("Records", []):
    bucket = record["s3"]["bucket"]["name"]
    key = urllib.parse.unquote_plus(record["s3"]["object"]["key"])

    parts = key.split("/")
    if len(parts) < 3:
        continue

    filename = parts[-1]
    note_id = filename.rsplit(".", 1)[0]
    user_id = parts[1]

    job_name = f"VoiceNote_{user_id}_{note_id}"
    file_uri = f"s3://{bucket}/{key}"
    ext = filename.rsplit(".", 1)[-1].lower()
    media_format = "webm" if ext == "webm" else ("mp3" if ext == "mp3" else "wav")
```

Why it is written this way

- **It loops over records.** S3 can batch more than one event in a single invocation. The loop handles each object.
- `unquote_plus` decodes URL-encoded key characters. S3 event keys are URL-encoded, so a key with spaces or special characters would otherwise be wrong.
- **The key shape is the contract.** `audio/user/note.webm` means the handler can parse ownership without a database call.
- **The job name is deterministic.** `VoiceNote_<userId>_<noteId>` lets the completion handler recover the same IDs. The `split('_', 2)` limit is important because user IDs contain hyphens, not underscores.
- **MediaFormat is explicit.** Transcribe needs to know how to decode the file. The frontend records WebM, but the mapping also accepts MP3 and WAV.
- **LanguageCode is en-US.** For a different audience this would become a parameter or a user setting.
- **Tags carry the user and note IDs.** Tags are useful for cost allocation, auditing, and tracing a job back to a note.

Why OutputBucketName is omitted

Transcribe can write its JSON output to a bucket you own, or to a service-managed bucket. This project omits `OutputBucketName` on purpose. AWS stores the result in a service-managed location, and the completion handler gets the download URL from `GetTranscriptionJob`. That removes an output bucket, an extra bucket policy, and extra cleanup from the template.

Why the work is asynchronous

- Transcription can take longer than an API request should. Starting a job returns immediately.
- The browser is not kept waiting. The UI shows a processing card and polls the notes API.
- If transcription fails, the failure is isolated to that job. It does not hold a Lambda open or block other uploads.

5.3 process_transcript.py - transcript to structured note

What it does

This function listens for the Transcribe completion event. It retrieves the job, downloads the transcript JSON, extracts the raw text, calls Bedrock to clean and structure it, and writes the finished note to DynamoDB.

```
detail = event.get("detail", {})
job_name = detail.get("TranscriptionJobName", "")

parts = job_name.split("_", 2)
if len(parts) < 3:
```

```
    return {"statusCode": 400, "body": "Invalid job name"}

user_id = parts[1]
note_id = parts[2]

job = transcribe_client.get_transcription_job(TranscriptionJobName=job_name)
transcript_uri = job["TranscriptionJob"]["Transcript"]["TranscriptFileUri"]
```

Why it is written this way

- **EventBridge gives a small event, not the transcript.** The handler uses the job name to ask Transcribe for the real result. This keeps events small and avoids passing large text through the event bus.
- **The job name is parsed defensively.** If the name does not have three parts, the function returns a 400-style response instead of crashing.
- **The transcript is downloaded with `urllib`.** The URI is a pre-signed URL from Transcribe. No extra SDK call or bucket permission is needed to read it.
- **The raw text is extracted from the nested JSON.** Transcribe returns `results.transcripts[0].transcript`. The code handles an empty result safely.

The AI call and the fallback

```
try:
    structured = structure_note_with_ai(raw_text)
    title = structured.get("title", "Voice Note")
    summary = structured.get("summary", "")
    cleaned_text = structured.get("cleanedText", raw_text)
    status = structured.get("status", "Planned")
    todos = structured.get("todos", [])
except Exception as ai_error:
    print(f"Bedrock structuring failed: {ai_error}")
    title = "Voice Note"
    summary = ""
    cleaned_text = raw_text
    status = "Planned"
    todos = []
```

- **Every field has a default.** If the model omits a key, the note still has a valid shape. The frontend does not need to handle missing fields.
- **The raw transcript is the fallback.** A failed AI call must not lose the user's recording. The note may be plain, but it exists.
- **The error is logged, not returned to the browser.** The browser is not waiting on this Lambda, so the log is the right place for diagnostics.

The DynamoDB write

```
table.put_item(Item={
    "userId": user_id,
    "noteId": note_id,
    "text": cleaned_text,
    "rawText": raw_text,
    "title": title,
    "summary": summary,
    "status": status,
    "todos": todos,
    "createdAt": now,
    "updatedAt": now,
    "audioKey": f"audio/{user_id}/{note_id}.{audio_ext}",
})
```

- **put_item is idempotent for this key.** If EventBridge delivers the same event twice, the same item is overwritten, not duplicated. That is a simple but effective retry safety net.
- **Both text and rawText are stored.** The cleaned version is what the user reads; the raw version is kept for traceability and re-processing.

- **Timestamps are UTC ISO-8601.** That is unambiguous and easy for the frontend to format.
- **audioKey is reconstructed.** The UI can refer to the original audio without a second database lookup or a separate table.

5.4 ai_notes.py - the shared AI module

What it does

This module owns the prompt, the Bedrock client, the model call, and the JSON parsing. It is shared by `process_transcript.py` and `notes_api.py`, so the same rules apply whether a note is created or edited.

The model and region

```
DEFAULT_MODEL_ID = os.environ.get("BEDROCK_MODEL_ID", "amazon.nova-micro-v1:0")
DEFAULT_REGION = os.environ.get("BEDROCK_REGION", "us-east-1")

_clients = {}

def _get_client(region_name):
    if region_name not in _clients:
        _clients[region_name] = boto3.client("bedrock-runtime", region_name=region_name)
    return _clients[region_name]
```

- **Nova Micro** is a small, fast, low-cost text model. The task is extraction and cleanup, not creative writing, so a large model is unnecessary.
- **us-east-1** is used because Nova Micro is not available in af-south-1. The rest of the stack stays in Cape Town; only the model call crosses regions.
- **The client cache** avoids creating a new boto3 client on every warm invocation. Lambda reuses the execution environment, and the module-level dictionary reuses the client.

The prompt contract

The prompt defines five jobs: remove filler words, create a short title, write one summary sentence, decide whether the note is Done or Planned, and extract actionable todos. It then gives the exact JSON shape. The last line says: no explanation, no markdown, just the JSON object.

- **Why a fixed schema:** the frontend and DynamoDB expect specific fields. Free-form prose would need another parsing layer and would be unreliable.
- **Why title max five words:** it keeps cards scannable and prevents the model from writing a paragraph as a title.
- **Why status is Planned or Done:** the UI has two sections. A small, closed set of values keeps filtering simple.
- **Why todos are extracted:** the product is not just transcription. The value is turning speech into actions.

The Bedrock call

```
response = bedrock.converse(
    modelId=model_id,
    messages=[{"role": "user", "content": [{"text": prompt}]}],
    inferenceConfig={"maxTokens": 1024, "temperature": 0.0},
)
text = response["output"]["message"]["content"][0]["text"].strip()
```

- **converse** is the model-agnostic Bedrock API. It handles the message format and lets the model be swapped without rewriting the request shape.
- **maxTokens 1024** is enough for a cleaned note and a short task list. It caps cost and keeps responses small.

- **temperature 0** makes the output as deterministic as possible. For structured extraction, variation is a bug, not a feature.
- **The fallback to** `invoke_model` is a compatibility path for environments where the converse API is unavailable. The primary path is converse.

The parser

```
if "```" in text:
    parts = text.split("```")
    for p in parts:
        p2 = p.strip()
        if p2.startswith("json"):
            p2 = p2[4:].strip()
        if p2.startswith("{") and p2.endswith("}"):
            text = p2
            break

try:
    return json.loads(text)
except Exception:
    start = text.find("{")
    end = text.rfind("}")
    if start != -1 and end != -1 and end > start:
        return json.loads(text[start:end + 1])
    raise
```

- **Models sometimes wrap JSON in markdown fences.** The parser strips the fences before parsing, so a cosmetic formatting choice does not break the pipeline.
- **The second attempt finds the outer JSON object.** If the model adds a sentence before or after the JSON, the parser can still recover it.
- **If both attempts fail, the exception propagates.** The caller decides how to degrade: process_transcript saves the raw text; notes_api keeps the edited text.

Why the AI is not called from the browser

- The browser would need AWS credentials, which cannot be kept secret.
- A malicious user could call the model directly and run up the bill.
- The prompt and output rules would be visible and editable.
- The backend can enforce the same schema for create and edit paths.

5.5 notes_api.py - the CRUD API

What it does

This one Lambda serves four routes: list notes, get one note, update a note, and delete a note. It identifies the user from the JWT, chooses the operation from the HTTP method and path, and returns JSON.

Reading the user and routing

```
authorizer = event.get("requestContext", {}).get("authorizer", {}).get("jwt", {})
claims = authorizer.get("claims", {})
return claims.get("sub") or claims.get("username") or "anonymous"

method = event.get("requestContext", {}).get("http", {}).get("method", "").upper()
note_id = (event.get("pathParameters") or {}).get("id")
```

- **The user ID always comes from the token.** The browser cannot choose whose notes to read or write.
- **The method comes from API Gateway's request context.** The route determines the operation. There is no separate endpoint for every action.
- **The note ID comes from the path.** For `/notes/{id}`, API Gateway places `id` in `pathParameters`.

List and get

```
if method == "GET" and not note_id:
    res = table.query(KeyConditionExpression=Key("userId").eq(user_id))
    return api_response(200, res.get("Items", []))

elif method == "GET" and note_id:
    res = table.get_item(Key={"userId": user_id, "noteId": note_id})
    item = res.get("Item")
    if not item:
        return api_response(404, {"error": "Note not found"})
    return api_response(200, item)
```

- **query, not scan.** A scan reads the whole table. A query reads one partition, which is faster and cheaper.
- **The key is always** `userId + noteId`**.** Even if someone guesses another user's note ID, the key still includes their own user ID, so the item is not found.
- **404 vs 500.** A missing note is a normal client outcome, not a server error.

Update and the reserved-word problem

```
update_parts = []
attr_names = {}
attr_values = {}

if "text" in body:
    update_parts.append("#t = :t, #ti = :ti, #su = :su, #st = :st, #td = :td")
    attr_names.update({"#t": "text", "#ti": "title", "#su": "summary",
                       "#st": "status", "#td": "todos"})

res = table.update_item(
    Key={"userId": user_id, "noteId": note_id},
    UpdateExpression="SET " + ", ".join(update_parts),
    ExpressionAttributeNames=attr_names,
    ExpressionAttributeValues=attr_values,
    ReturnValues="ALL_NEW",
)
```

- **Why not string-format the values into the expression:** that would be injection-prone and would break on quotes or special characters. Values are passed separately.
- **Why #st and friends:** DynamoDB reserves words such as `status`. Expression attribute names let the code use a safe placeholder that maps to the real attribute name.
- **Why build the expression dynamically:** the client may update only text, only status, or only todos. The function updates only the fields that were sent.
- **ReturnValues ALL_NEW** returns the updated item, so the frontend can render the new state without a second GET.

Editing text re-runs the AI

When the request contains `text`, the backend calls the same `structure_note_with_ai` function used by the pipeline. It updates the cleaned text, title, summary, status, and todos together. That keeps the note internally consistent: the user edits the transcript, and the derived fields are regenerated from the new transcript.

- If the AI call fails, only the text is updated. The existing title, summary, and todos are left alone.
- If the request also contains status or todos, the code checks the attribute-name map so the AI values are not overwritten by a stale manual value.

Todo normalisation

```
for todo in body["todos"] or []:
    if isinstance(todo, str):
        todos.append({"text": todo, "done": False})
    elif isinstance(todo, dict):
        todos.append({
            "text": str(todo.get("text", "")),
            "done": bool(todo.get("done", False)),
        })
```

- The API accepts both the old string format and the current object format. That makes migrations and cached clients tolerant.
- Every todo is stored as `{ text, done }`, so the frontend has one consistent shape to render.
- Missing or malformed fields are coerced to safe defaults instead of crashing the update.

Delete and response helper

- `delete_item` removes the exact user-and-note key. A user cannot delete another user's note.
- The helper always sets `Content-Type: application/json` and a CORS origin header, so the browser can read the response.
- Unsupported routes return 400; unexpected exceptions return 500 with the error message. In production I would log the stack trace and return a generic message.

6. Frontend code, file by file

The frontend has two jobs: make the recording flow feel instant, and never expose more than a public client ID. The code is organised around services for auth, API, and cache, and components for the screens.

6.1 App.tsx and ProtectedRoute.tsx - routing and guards

- The app is wrapped in `ThemeProvider` and `AuthProvider`, then `BrowserRouter`.
- Public routes are `/login` and `/signup`. The notes list and detail routes are wrapped in `ProtectedRoute`.
- ProtectedRoute waits for `loading` to become false before deciding. Without that wait, a signed-in user would be redirected to login for a split second while the session is restored.
- If there is no user after loading, it renders `<Navigate to="/login" replace />`. The `replace` flag keeps the protected URL out of the back-button history.

6.2 AuthContext.tsx - session lifecycle

What it does

This context owns the current user, restores the session on page load, refreshes tokens before they expire, syncs auth changes across tabs, and clears the notes cache on logout.

The expiry check

```
function getTokenExpiry(token) {
  const payloadBase64 = token.split(".")[1];
  const decoded = JSON.parse(atob(payloadBase64.replace(/-/g, "+").replace(/_/g, "/")));
  return decoded.exp ?? 0;
}

function isExpiringSoon(token) {
  const expiresAt = getTokenExpiry(token);
  if (!expiresAt) return false;
  return Date.now() / 1000 > expiresAt - 300;
}
```

- **JWTs are base64url encoded.** The code converts `-` to `+` and `_` to `/` before calling `atob`.
- **The exp claim is in seconds.** JavaScript time is in milliseconds, so the comparison divides by 1000.
- **The five-minute window** refreshes before the token is actually expired. That avoids a failed request at the exact boundary.
- **If the token is malformed, the function returns 0 and does not force a refresh.** The normal 401 path will handle a genuinely bad token.

Cross-tab sync and logout

- The `storage` event fires when another tab changes localStorage. The context updates the user to match.
- The API layer dispatches a custom `auth:logout` event when a refresh fails. The context clears the notes cache and sets the user to null.
- This keeps multiple tabs consistent without a global state library.

6.3 cognitoAuth.ts - talking to Cognito

- The app calls the Cognito user pool endpoint directly with the JSON protocol: `Content-Type: application/x-amz-json-1.1` and an `X-Amz-Target` header.
- `signUp` and `confirmSignUp` create the account and verify the email code.

- `signIn` uses `USER_PASSWORD_AUTH` and returns the access, ID, and refresh tokens.
- `refreshSession` uses `REFRESH_TOKEN_AUTH` and returns a new access token and ID token. The refresh token itself is reused.
- **Why no secret:** the app client is public. The client ID is not a secret; it is an identifier. The security comes from the user's credentials, TLS, and the token lifetimes.
- **Why mock tokens:** when `VITE_COGNITO_CLIENT_ID` is absent, the module returns fake tokens so the UI can run without a deployed backend. This is a development convenience, not a production path.

6.4 api.ts - auth header, refresh, and API calls

What it does

This service wraps every API call. It attaches the Bearer token, handles a 401 by refreshing once and retrying, and converts the API note shape into the UI note shape.

The retry logic

```
async function apiFetch(url, options = {}) {
  const response = await fetch(url, {
    ...options,
    headers: { ...buildAuthHeader(), ...(options.headers || {}) },
  });

  if (response.status === 401) {
    const newIdToken = await tryRefreshToken();
    if (newIdToken) {
      return fetch(url, {
        ...options,
        headers: { Authorization: `Bearer ${newIdToken}`, ...(options.headers || {}) },
      });
    }
  }
  return response;
}
```

- **Why only one retry:** if the refreshed token is also rejected, looping would hammer Cognito and hide the real problem. One retry handles the normal expiry case.
- **Why refresh on 401 and not only on a timer:** the timer is an optimisation; the 401 is the source of truth. The server decides whether a token is still valid.
- **Why the StorageEvent is dispatched manually:** the `storage` event only fires in other tabs. The current tab needs to update AuthContext too, so the service dispatches a synthetic event.
- **If refresh fails:** the auth record is removed and `auth:logout` is dispatched. The UI returns to the login screen instead of staying in a broken state.

The API calls

- `getUploadUrl` posts the extension and returns the URL, note ID, and key.
- `uploadAudioToS3` does a raw `PUT` to the signed URL with `Content-Type: audio/webm`. It does not add the Authorization header; the signature is in the URL.
- `fetchNotes` calls `GET /notes`. `updateNote` and `deleteNote` use the note ID in the path.
- `fetchAndCacheNotes` fetches, maps, and writes the cache in one place. The UI does not need to know about both layers.

Mapping API data to UI data

- The API returns `noteId`, `text`, and optional fields. The UI uses `id`, `context`, and a normalised todo list.
- The mapper supplies a fallback title from the first sentence if the model did not return one.

- It normalises old string todos and new `{ text, done }` todos into one shape, so the components stay simple.

6.5 notesCache.ts - instant rendering

- The cache is a localStorage copy of the last known notes list.
- `getCachedNotes` parses it safely and returns an empty list on any error.
- `setCachedNotes` writes the whole list.
- `updateCachedNote` and `removeCachedNote` keep the cache in sync after edits and deletes.
- **Why cache at all:** the notes page can render immediately on a cold start or a slow network, then refresh from the API in the background. That is the stale-while-revalidate pattern.
- **Why localStorage and not IndexedDB:** the data is small JSON. localStorage is synchronous and simple. IndexedDB would be overkill for a list of notes.

6.6 VoiceRec.tsx - recording and upload

What it does

This component asks for microphone permission, records WebM audio, collects chunks, releases the microphone, and uploads the Blob through the presigned URL.

```
const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
const recorder = new MediaRecorder(stream, { mimeType: "audio/webm" });

recorder.ondataavailable = (e) => {
  if (e.data.size > 0) audioChunksRef.current.push(e.data);
};

recorder.onstop = async () => {
  stream.getTracks().forEach((track) => track.stop());
  if (!cancelledRef.current) {
    const audioBlob = new Blob(audioChunksRef.current, { type: "audio/webm" });
    await uploadAudio(audioBlob);
  }
  cancelledRef.current = false;
};
```

- **getUserMedia** is the browser API that requests microphone access. The user must grant permission; the code handles the rejection with a visible error message.
- **MediaRecorder** encodes the stream. WebM is widely supported in Chrome and Edge, which is enough for this project.
- **ondataavailable** can fire multiple times. The chunks are collected and combined into one Blob at the end.
- **onstop releases the microphone tracks.** Without that, the browser keeps the recording indicator on and the device stays busy.
- **cancelledRef** separates save from cancel. Both actions stop the recorder, but only save uploads. This avoids duplicating the stop logic.

The upload sequence

- Set `uploading` true and show a status message.
- Call `getUploadUrl("webm")` to get the signed URL.
- PUT the Blob to S3 with the matching content type.
- Show "Processing in AWS..." and call `onUploadSuccess` so the notes page can start polling.
- On failure, show the error and reset the uploading state. The audio is not silently lost without feedback.

6.7 NotesPage.tsx - the list and the processing state

- **Cache first:** on mount, it reads cached notes and renders them immediately, then calls the API.
- **Known IDs:** a ref keeps the set of note IDs already seen. When a new note appears, the matching processing card is removed.
- **Pending cards:** after an upload, a temporary card is added. The list polls every five seconds for up to ninety seconds.
- **Why poll:** the upload request returns before Transcribe and Bedrock finish. Polling is a simple way to show the result without WebSockets or push notifications.
- **Why a timeout:** if processing fails or takes too long, the UI stops polling and clears the placeholder instead of polling forever.
- **Pull-to-refresh:** touch events at the top of the page track the drag distance and trigger a refresh past a threshold. It is a mobile-friendly manual refresh.
- **Optimistic todo toggle:** the UI updates the todo and status immediately, writes the cache, then calls the API. If the API fails, it restores the previous note and shows an error. This makes the app feel fast without lying permanently.
- **Delete confirmation:** deletion opens a modal and only calls the API after confirmation. The item is removed from both state and cache.

6.8 NotesDetail.tsx - one note

- It finds the note in the cache by ID and renders immediately. If the note is missing, it shows a not-found state.
- Edit mode loads the transcript into a textarea. Saving sends the new text; the backend re-runs the AI and returns the updated note.
- The status toggle sends only the new status. The backend updates that field without touching the transcript.
- Todo toggles use the same optimistic pattern as the list, with rollback on failure.
- Delete asks for confirmation, removes the cache entry, and navigates back to the list.
- The date is formatted with the `en-ZA` locale, which matches the intended South African audience.

6.9 ThemeContext.tsx and NoteCard.tsx - the small pieces

- ThemeContext stores `dark` or `light` in localStorage and sets `data-theme` on the document element. CSS variables respond to that attribute, so the whole app switches without prop drilling.
- NoteCard is presentational. It renders the status, title, context, and todos.
- The todo button calls `preventDefault` and `stopPropagation`. The card is wrapped in a link, so without this a todo click would also navigate to the detail page.

7. Questions you should be able to answer

These are the questions that separate "I followed a tutorial" from "I understand my system". Answer each one in two or three sentences, using the code as evidence.

Q1. Why did you upload directly to S3 instead of sending audio to Lambda?

Because audio is binary and can be large. A presigned URL lets the browser write one object directly to S3, which avoids API Gateway and Lambda payload limits and keeps the upload fast. The Lambda only signs the URL, so it never handles the audio.

Q2. What is SigV4 and why is it in the code?

SigV4 is AWS Signature Version 4, the signing protocol for AWS requests. Boto3 uses the Lambda's IAM credentials to sign the PUT request and puts the signature in the URL. The browser can then upload without holding AWS credentials. I force s3v4 and virtual addressing so the signature and regional endpoint are correct.

Q3. Why does the presigned URL expire after 300 seconds?

Five minutes is enough for a phone to start and finish the upload, and it limits how long a leaked URL can be reused. The URL is also scoped to one object and one operation.

Q4. Why does the frontend send Content-Type: audio/webm?

The content type is one of the signed headers. If the browser sends a different value from the one used to generate the URL, S3 rejects the signature. Matching it is part of making the signed request valid.

Q5. Why use Cognito instead of storing users yourself?

Cognito gives managed sign-up, email verification, password hashing, JWT signing, and refresh tokens. Writing that yourself is a security risk and a lot of code. It also integrates directly with API Gateway's JWT authorizer.

Q6. Why is there no client secret in the frontend?

A browser cannot keep a secret. The app client is public by design. Security comes from the user's password over TLS, short-lived tokens, and the authorizer. A client secret in a SPA would be visible to anyone.

Q7. Why does the API use the ID token and not the access token?

The HTTP API authorizer is configured with the app client ID as the audience, and the ID token's aud claim matches that client. The ID token also carries the user claims such as sub and email that the Lambda needs.

Q8. How do you stop user A from reading user B's notes?

The user ID is read from the validated JWT, never from the request body. Every DynamoDB key starts with that user ID. Even if user A guesses user B's note ID, the get or update uses user A's partition and returns not found.

Q9. Why is userId the partition key?

The main access pattern is "list my notes". A query on userId reads one partition and returns only that user's notes. It is also the natural tenant boundary.

Q10. Why query instead of scan?

A scan reads every item in the table and then filters. A query reads one partition using the key. Query is faster, cheaper, and scales with the user's notes rather than the whole table.

Q11. Why do you use expression attribute names like #st?

DynamoDB reserves words such as status. Expression attribute names map a safe placeholder to the real attribute, so the update expression is valid. Values are passed separately in ExpressionAttributeValues, which also avoids injection.

Q12. Why is the note ID generated on the server?

The client should not choose a key. Server-side UUIDs prevent overwriting another note, make guessing impractical, and give the S3 key and DynamoDB item a stable identity.

Q13. Why does the job name contain the user and note IDs?

The Transcribe completion event contains the job name. Encoding the IDs lets process_transcript recover the owner and note without another database lookup. The split is limited to two underscores because the IDs contain hyphens, not underscores.

Q14. Why is OutputBucketName omitted?

Transcribe can write to a service-managed bucket. The completion handler gets the transcript URL from GetTranscriptionJob. That removes an output bucket, its policy, and its cleanup from the template.

Q15. Why is transcription asynchronous?

Audio transcription can take longer than an API request should. Starting a job returns immediately, and EventBridge tells us when it is done. The UI shows a processing card instead of holding a request open.

Q16. What happens if Bedrock fails?

process_transcript catches the error and saves the raw transcript with safe defaults. The note is still created, so the user does not lose the recording. notes_api does the same for edits: it updates the text without replacing the derived fields.

Q17. Why use temperature 0?

The task is structured extraction, not creative writing. Temperature 0 reduces variation and makes the JSON output more consistent. I also ask for a fixed schema and parse defensively.

Q18. Why parse markdown code fences?

Models sometimes wrap JSON in triple backticks even when asked not to. The parser strips the fences and, if needed, extracts the outer JSON object. That makes the pipeline resilient to a cosmetic model behaviour.

Q19. Why use Nova Micro?

It is a small, fast, low-cost model, and the task is cleanup and extraction. A larger model would add cost and latency without improving the result much. It runs in us-east-1 because it is not available in af-south-1.

Q20. Why is the Bedrock client cached in a module-level dictionary?

Lambda reuses warm execution environments. Creating the boto3 client once and reusing it avoids repeated client setup on every invocation. The dictionary also supports more than one region if that is added later.

Q21. Why does the frontend poll instead of using WebSockets?

The processing time is short and the update is simple. Polling every five seconds for up to ninety seconds is much less infrastructure than a WebSocket API, connection management, and reconnect logic. It is a deliberate simplicity trade-off.

Q22. Why cache notes in localStorage?

It gives an instant first render and a usable list on a slow network. The app then refreshes from the API in the background. It is a stale-while-revalidate pattern, not the source of truth.

Q23. Why optimistic updates for todos?

Ticking a task should feel instant. The UI updates immediately, writes the cache, and calls the API. If the call fails, it rolls back to the previous state and shows an error, so the user is never left with a silent inconsistency.

Q24. Why is CORS set to *?

It keeps the development and demo setup simple. For production I would restrict it to the actual frontend origin. The API is still protected by the JWT authorizer, but a tighter origin reduces abuse surface.

Q25. Why use ARM64 Lambdas?

Graviton is cheaper for this workload and Python runs on it without changes. The functions are light, so the price difference is small but free to take.

Q26. Why use HTTP API instead of REST API?

HTTP API is cheaper and simpler, and it has native JWT authorisation. This project does not need REST-only features such as usage plans, request validators, or API keys.

Q27. Why is the S3 notification a custom resource?

The bucket notification, the Lambda permission, and the function create a circular dependency in a normal SAM S3 event. The custom resource applies the notification after the permission exists, which breaks the cycle. It also clears the notification on delete.

Q28. What would you improve with more time?

Restrict CORS, add rate limiting and WAF, use SRP or the hosted UI instead of USER_PASSWORD_AUTH, add retries and a dead-letter queue, add integration tests, add structured logging and alarms, and add a GSI if notes need chronological ordering at scale.

8. Security design and honest limitations

You do not need to claim the project is bank-grade. You need to show that you made deliberate security choices and that you know what is missing. That is more credible than pretending there are no trade-offs.

What is protected, and how

Risk	Control in the project	Where it lives
Unauthenticated API access	Cognito JWT authorizer on every route	template.yaml, HttpApi.Auth
Reading another user's notes	userId from the token is the partition key	notes_api.py, get_upload_url.py
AWS credentials in the browser	Presigned S3 URL instead of access keys	get_upload_url.py
Leaked upload URL	300-second expiry and one-object scope	ExpiresIn=300
Large uploads through the API	Browser uploads directly to S3	VoiceRec.tsx, uploadAudioToS3
Unbounded storage cost	S3 lifecycle expires audio after 7 days	template.yaml
Public audio access	Private bucket, no public read policy	template.yaml
Over-privileged functions	Per-function IAM policies	template.yaml
Prompt or model abuse from the client	Bedrock is only called from Lambda	ai_notes.py
Lost data when AI fails	Raw transcript fallback	process_transcript.py

The honest limitations

- **CORS is open.** `AllowedOrigins: ['*']` is fine for a demo, but production should list the exact frontend origin.
- **USER_PASSWORD_AUTH is simple, not ideal.** It sends the password to Cognito over TLS. SRP or the hosted UI is stronger because the app never handles the password directly.
- **There is no rate limiting or WAF.** A determined user could call the API repeatedly. API Gateway throttling, WAF, and per-user quotas would be the next step.
- **There is no dead-letter queue.** If process_transcript fails repeatedly, the event is retried by the event source, but there is no DLQ for inspection.
- **There is no virus scanning.** Audio is uploaded by authenticated users, but a production system should scan untrusted uploads.
- **Tokens are in localStorage.** That is common for a SPA, but it is accessible to JavaScript. A stricter design uses httpOnly cookies or a backend-for-frontend.
- **There is no pagination.** `GET /notes` returns all notes for the user. For a large account, add pagination and a `createdAt` sort key or GSI.
- **There is no formal test suite.** The project is verified manually and with build and lint checks. Integration tests for the upload-to-note path would be the next quality step.

How to talk about trade-offs without sounding unsure

> *Use the pattern: "I chose X because Y. The trade-off is Z. In production I would address it by ...". That shows judgement. For example: "I used a public app client with USER_PASSWORD_AUTH to keep the demo simple. The trade-off is that the app handles the password. In production I would move to SRP or the hosted UI."*

9. The 5-minute video script

Do not read this word for word. Use it as the order of your explanation. Each block has the point you must land, and a few lines you can say almost exactly. Keep the code open on the screen and point to the file or function as you speak.

Time	Section	What the viewer should understand
0:00 - 0:20	Opening	What the app does and the one-sentence architecture.
0:20 - 0:50	Demo	Record, upload, processing, note appears, open and edit.
0:50 - 1:35	Architecture	The services and the direction of the arrows.
1:35 - 2:15	Upload and SigV4	Why direct upload and what the signed URL proves.
2:15 - 3:00	Transcribe and Bedrock	Async job, EventBridge, structured JSON, fallback.
3:00 - 3:40	Data and API security	userId partition key, query not scan, safe updates.
3:40 - 4:25	Frontend	Token refresh, cache, MediaRecorder, polling, optimistic UI.
4:25 - 5:00	Close	Trade-offs, what you would improve, final summary.

0:00 - 0:20 | Opening

- "This is VoiceToNote. It turns a spoken note into a clean, structured task list."
- "The user records audio in the browser. It goes to S3, then Transcribe, then Bedrock, then DynamoDB."
- "I built it serverless on AWS, so there is no server running when nobody is using it."
- Show the notes page, not the code yet. Let the product land first.

0:20 - 0:50 | Demo

- Sign in, press Rec, say a short note with a task in it, press Stop.
- "While it processes, the UI shows a placeholder card and polls the API every five seconds."
- Wait for the real note, open it, point out title, summary, tasks, and transcript.
- "I can edit the transcript, tick tasks, change status, or delete the note."
- Keep this under thirty seconds. The demo is proof, not the explanation.

0:50 - 1:35 | Architecture

- "The frontend is React and Vite. Cognito handles sign-up and login and returns a JWT."
- "API Gateway validates that JWT before any Lambda runs. That means unauthenticated requests never reach my code."
- "There are four Lambdas: one signs the upload URL, one starts transcription, one processes the transcript, and one serves the notes API."

- "S3 stores the audio. Transcribe converts speech to text. EventBridge tells the processing Lambda when the job is done. Bedrock structures the text, and DynamoDB stores it per user."
- Point to the architecture diagram or the template file. Name the services in the order the data moves.
- "The important thing is the direction: the browser only talks to the API and to one short-lived S3 URL."

1:35 - 2:15 | Upload and SigV4

- Open `get_upload_url.py`.
- "Here is the upload function. It does not take a userId from the request body."
- "It reads the sub claim from the JWT that API Gateway already validated, generates a UUID, and builds the key audio/user/note.webm."
- "Then it creates a presigned S3 URL with a 300-second expiry."
- "SigV4 is AWS Signature Version 4. It is how AWS proves a request is authentic and unchanged. The Lambda signs the request with its IAM role, and the signature goes into the URL."
- "That means the browser never receives AWS credentials. The URL can only PUT one object, and only for five minutes."
- "I force s3v4 and virtual addressing so the signature and regional endpoint are correct. Content-Type is signed too, so the frontend must send audio/webm exactly."
- If asked why not upload through Lambda: "Audio is binary and can be large. Direct upload avoids payload limits and keeps the Lambda out of the data path."

2:15 - 3:00 | Transcribe and Bedrock

- Open `start_transcription.py`.
- "The S3 upload triggers this function. It reads the key, recovers the user and note IDs, and starts an asynchronous Transcribe job."
- "The job name encodes the same IDs, so the completion handler can find them again."
- "I omit OutputBucketName, so AWS manages the transcript output. The next function gets the URL from GetTranscriptionJob."
- Open `process_transcript.py` and then `ai_notes.py`.
- "EventBridge invokes this function when Transcribe completes. It downloads the transcript JSON and calls Bedrock Nova Micro through the converse API."
- "I use temperature zero and ask for JSON only, because this is extraction, not creative writing."
- "If Bedrock fails, I still save the raw transcript. The note is less polished, but the user's data is not lost."
- "The same AI module is used when the user edits the transcript, so the title, summary, status and todos stay consistent."

3:00 - 3:40 | Data and API security

- Open `notes_api.py` and the DynamoDB table in `template.yaml`.
- "The table uses userId as the partition key and noteId as the sort key."
- "The userId always comes from the JWT, so every query is scoped to one user. User A cannot read user B's notes even if they guess a note ID."
- "I use query, not scan, because query reads one partition and does not touch the whole table."

- "Updates use UpdateExpression with expression attribute names and values. That avoids reserved words like status and avoids string-formatting user input into the expression."
- "Editing the transcript re-runs Bedrock, so the derived fields are regenerated from the new text. Ticking a todo only updates todos and status."

3:40 - 4:25 | Frontend

- Open `AuthContext.tsx` and `api.ts`.
- "The app stores the Cognito tokens in localStorage and refreshes them five minutes before expiry by decoding the exp claim in the JWT."
- "apiFetch attaches the Bearer ID token. If the API returns 401, it refreshes once and retries. If refresh fails, it clears the session and sends the user back to login."
- Open `VoiceRec.tsx`.
- "VoiceRec uses getUserMedia and MediaRecorder. It collects chunks, releases the microphone on stop, and uploads the Blob to the presigned URL."
- Open `NotesPage.tsx`.
- "After upload, the page shows a processing card and polls for up to ninety seconds. Notes are cached in localStorage so the list renders instantly, then refreshes from the API."
- "Todo toggles are optimistic: update the UI, call the API, and roll back if it fails."

4:25 - 5:00 | Close

- "The main trade-off is simplicity versus production hardening."
- "CORS is open and the app client uses USER_PASSWORD_AUTH for the demo. With more time I would restrict CORS, use SRP or the hosted UI, add rate limiting and WAF, and add retries and a dead-letter queue."
- "But the core design is sound: least-privilege IAM, short-lived signed URLs, user-scoped data, event-driven processing, and graceful AI failure."
- "That is VoiceToNote."
- Stop talking at five minutes. Do not add a new topic in the last ten seconds.

Five lines to memorise

1. The browser uploads directly to S3 with a five-minute SigV4 presigned URL, so no AWS credentials touch the client.
2. API Gateway validates the Cognito JWT before any Lambda runs, and every database key is derived from the token's sub claim.
3. S3 triggers Transcribe, EventBridge triggers processing, and Bedrock turns the transcript into structured JSON.
4. DynamoDB is partitioned by userId, so tenant isolation is enforced at the data-model level.
5. If AI fails, the raw transcript is still saved, so the system degrades gracefully instead of losing data.

Phrases to use, and phrases to avoid

Use this	Avoid this
"The browser uploads directly to S3 with a signed URL."	"The file just goes to the cloud."
"API Gateway validates the JWT before Lambda runs."	"The frontend handles security."
"The user ID comes from the token, not the request body."	"I pass the user ID from the frontend."
"Transcribe is asynchronous and EventBridge signals completion."	"The app waits for the AI."
"If Bedrock fails, the raw transcript is still saved."	"The AI always works."
"This is a deliberate trade-off for the demo."	"It is not production-ready, but whatever."

10. Last-minute cheat sheet

Numbers and settings

Item	Value	Why it matters
Presigned URL expiry	300 seconds	Short replay window, enough for a phone upload.
S3 audio lifecycle	7 days	Cost and privacy; note text stays in DynamoDB.
DynamoDB billing	PAY_PER_REQUEST	No idle capacity, automatic scaling.
Lambda memory	256 MB	Enough for boto3, JSON, and model responses.
Lambda timeout	30 s default, 60 s for AI paths	Long enough for Transcribe and Bedrock calls.
Lambda architecture	arm64	Graviton is cheaper for this workload.
Python runtime	3.12	Current supported runtime.
Bedrock model	amazon.nova-micro-v1:0	Small, fast, low-cost extraction model.
Bedrock region	us-east-1	Nova Micro is not available in af-south-1.
Model tokens	1024 max	Caps cost and response size.
Temperature	0.0	Deterministic structured output.
Polling interval	5 seconds	Simple progress without WebSockets.
Polling timeout	90 seconds	Stops the placeholder if processing fails.
Transcribe language	en-US	Matches the demo audio.

File map

File	Role
template.yaml	All AWS infrastructure: Cognito, S3, DynamoDB, API, Lambdas, triggers, IAM.
backend/src/get_upload_url.py	Validates the user from the JWT and signs the S3 upload URL.
backend/src/start_transcription.py	S3 event handler; starts an async Transcribe job.
backend/src/process_transcript.py	Transcribe completion handler; downloads, structures, saves.
backend/src/ai_notes.py	Shared Bedrock prompt, call, client cache, and JSON parser.
backend/src/notes_api.py	List, get, update, and delete notes for the authenticated user.
frontend/src/services/cognitoAuth.ts	Sign-up, confirm, sign-in, refresh against Cognito.

File	Role
frontend/src/services/api.ts	Bearer header, 401 refresh, API calls, S3 upload, note mapping.
frontend/src/services/notesCache.ts	localStorage cache for instant rendering.
frontend/src/context/AuthContext.tsx	Session restore, expiry refresh, cross-tab sync, logout.
frontend/src/Components/VoiceRec.tsx	Microphone capture, MediaRecorder, upload flow.
frontend/src/Pages/NotesPage.tsx	List, search, filters, polling, optimistic todos, delete.
frontend/src/Pages/NotesDetail.tsx	One note: edit transcript, toggle status, delete.

Deployment and environment

```
# Backend
cd /home/paseka-dev/VoiceToNote
sam build
sam deploy

# Frontend
cd frontend/VoiceToNote
npm install
npm run dev
npm run build
```

The frontend reads `VITE_API_URL`, `VITE_AWS_REGION`, and `VITE_COGNITO_CLIENT_ID`. The backend reads `NOTES_TABLE`, `AUDIO_BUCKET`, `TRANSCRIBE_REGION`, and `BEDROCK_MODEL_ID` from Lambda environment variables.

Deployed stack outputs

```
Stack:           voice-to-note
Region:          af-south-1
ApiUrl:          https://0drykdpyx3.execute-api.af-south-1.amazonaws.com
UserPoolId:      af-south-1_IPdMS0Uyh
UserPoolClientId: f8tuan8h0v7e4uerm9h9t5pa2
AudioBucketName:  voice-to-note-audiobucket-hmz11yqbzipo
```

The 60-second recap

> *Cognito authenticates the user. API Gateway validates the JWT. A Lambda signs a five-minute S3 URL. The browser uploads audio directly. S3 triggers Transcribe. EventBridge triggers processing. Bedrock cleans and structures the transcript. DynamoDB stores it under the user's partition. The React app caches, polls, and renders. Every permission is narrow, every URL expires, and every AI failure still leaves the user with their data.*

End of document